# 基于 RISC-V 与 FPGA 异构架构的

# 2D 图形渲染加速引擎

**技术文档**

2026 年全国大学生嵌入式芯片与系统设计竞赛 · FPGA 赛道

**基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎**

**技术文档**

**2026 年全国大学生嵌入式芯片与系统设计竞赛 · FPGA 赛道**

# 摘要

本作品面向「基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎」赛题，在易灵思 Titanium Ti60F225 平台上实现了一套以 AXI4 BitBlt 引擎为核心的异构 2D 渲染系统。RISC-V（Sapphire SoC）承担场景逻辑与高层决策，FPGA 侧完成像素搬运、Alpha 混合、颜色键控与 1080p60 扫描输出。

在实现赛题基础要求（硬件 BitBlt、1080p60 输出）之后，本作品进一步定位并消除了异构渲染链路中的串行瓶颈：CPU 命令提交与精灵状态维护。基准实现在 32 个精灵时需 256 次寄存器写与 32 次轮询，且与硬件执行完全不重叠，成为系统吞吐的实际限制。本作品沿数据通路逐层解决这一问题，形成三项核心设计。

第一，精灵调度硬件化。将精灵的遍历、位置积分、边界处理、擦除与绘制命令生成全部迁移至硬件表遍历器。CPU 仅在场景初始化时写入一次精灵表，此后每帧只发送一个启动信号，对精灵的寄存器操作降为零，实现软硬件真正并行。

第二，片内精灵位图缓存。引擎内置四槽直接映射的精灵位图缓存，命中时以 17 个周期从片内 BRAM 取数，替代每次回 DDR 所需的 650 个周期。700 个精灵规模的渲染时间由 45 ms 降至 0.92 ms，提升 49 倍，实测命中率 75%。

第三，访存事务合并。针对紧凑存储的精灵位图，将「每扫描行一次短突发」重构为「整块一次长突发」，把多个 AXI 读事务合并为单次传输，使总线有效带宽利用率由 33% 提升至 88%（2.66 倍），64×64 位图场景达到 98%。

系统支持双缓冲、Alpha 混合、颜色键控、硬件裁剪与片上可观测性。实测指标为：1500 个 8×8 精灵稳定运行于 59.2 fps，等效像素量 96000；256 个 16×16 弹幕单帧渲染 0.95 ms，占 60 fps 帧预算的 5.7%；交互游戏场景在约 180 个同屏活动对象下保持 59.2 fps 满帧。全部内部状态可由片内监控一次读出，为「硬件搬运正确性」提供直接证据。

# 一、作品概述

## 1.1 作品简介

本作品是一套完整的异构 2D 图形渲染链路，运行于易灵思 Titanium Ti60F225 开发平台。系统由 RISC-V 主控、2D 渲染加速引擎、DDR3 帧缓冲与 HDMI 扫描输出四部分组成，通过 AXI4 总线互联。

渲染加速引擎作为 AXI4 从设备接收命令与参数，同时作为 AXI4 主设备直接读写 DDR3 帧缓冲，支持纯色填充、位块拷贝、Alpha 混合、颜色键控与混合位块传输五种绘制操作。引擎内部含命令 FIFO、精灵表遍历器、位图缓存与三条渲染通路，可在 1080p60 分辨率下以硬件完成全部像素搬运。

在满足赛题基础要求之外，本作品针对异构架构的典型瓶颈展开优化：CPU 与加速器之间的串行依赖、精灵位图的重复读取、以及短突发对总线带宽的浪费。三项优化分别对应调度层、缓存层与传输层，形成一套可复用的异构渲染优化方法。

## 1.2 功能与特性

基础 BitBlt 能力。硬件直接操作 DDR3 帧缓冲，支持五种绘制操作：纯色填充（FILL）、位块拷贝（COPY）、Alpha 混合（ALPHA）、颜色键控（COLOR KEY）与混合位块传输（BLIT，键控与 Alpha 同时生效）。命令格式包含操作类型、源地址、目的地址、宽度、高度、源行距、目的行距、全局 Alpha 与键控色共 11 个字段。

1080p60 视频输出。采用 HDMI TMDS 编码，16 路行预取缓冲，双帧缓冲切换消除画面撕裂。扫描输出从帧缓冲按标准 1080p60 时序读出行数据，通过 AXI4 主设备端口与渲染引擎共享总线仲裁。

精灵表硬件遍历。CPU 一次写入精灵表（位置、速度、透明标志），硬件按帧展开为擦除与绘制命令，并在绘制完成后回写新位置，支持边界处理。该特性使 CPU 每帧对精灵的操作为零。

硬件裁剪。越界精灵按可见区域裁剪，支持精灵部分离开屏幕仍正确绘制，而非只能整体反弹。

片上可观测性。11 项运行时异常状态（握手超时、响应错误、遍历器停滞、扫描输出欠载等）各配粘滞位与饱和计数器，软件通过一条命令即可读出全部内部状态，为调试与验证提供直接证据。

演示能力。系统内置同屏精灵压力测试场景、CPU 渲染与硬件渲染实时帧率对比、以及交互式弹幕射击游戏演示。

## 1.3 主要技术特点

AXI4 主从双角色架构。加速器作为 AXI4 从设备挂载于 Sapphire SoC 总线，接收命令与参数；同时作为 AXI4 主设备直接读写 DDR3 帧缓冲。双角色设计使引擎既能被 CPU 精确控制，又能独立完成大批量像素搬运。

352 位命令记录。一条绘制命令压缩为 44 字节的定长记录，经命令 FIFO 批量下发。相比逐字段寄存器写，该设计将 CPU 提交开销降低一个数量级。

三处理通路并行。命令分发器按段把位块拆分为 16 像素宽的段，三条渲染通路轮流处理，隐藏 DDR 往返延迟。

精灵位图片内缓存。四槽直接映射，以位图地址高位做槽位索引，命中时直接从 BRAM 复制，避免每条命令重读 DDR。

扫描输出行预取。16 路 bank × 128 位宽，活动区提前 10 行预取，消隐期加深至 14 行，保证帧首不欠载。

完整可观测性。7 个监控寄存器与 11 项条件，配合粘滞位与饱和计数器，单次 dump 即可获得全部运行时状态。

## 1.4 主要性能指标

| **指标项** | **实测数值** |
|---|---|
| 开发平台 | 易灵思 Ti60F225I3 + Sapphire RISC-V SoC |
| 总线接口 | AXI4（主从双角色） |
| 显示输出 | 1920 × 1080 @ 60 Hz HDMI（TMDS） |
| 同屏精灵数量 | 1500 个 8×8 精灵 @ 59.2 fps |
| 等效像素吞吐 | 96000 像素/帧 |
| 单帧渲染延迟 | 256 个 16×16 弹幕 0.95 ms |
| 帧预算占用 | 5.7%（60 fps 帧预算 16.7 ms） |
| 位图缓存命中率 | 75% |
| 总线有效带宽利用率 | 16×16 位图 88%；64×64 位图 98% |
| CPU 每帧精灵操作 | 0 次 |
| 扫描输出稳定性 | 欠载计数恒定，显示帧与生成帧 1:1 |

## 1.5 主要创新点

创新点一：精灵调度硬件化。将精灵遍历、位置积分、边界处理与命令生成全部移入硬件表遍历器。CPU 每帧仅下发一次启动信号，对精灵的寄存器操作为零。相比基准实现的「每精灵 8 次寄存器写 + 每帧轮询」，该设计从调度层面消除了 CPU 与加速器的串行依赖。

创新点二：片内精灵位图缓存。四槽直接映射位图缓存，命中时 17 个周期完成取数（DDR 路径需 650 个周期）。在 700 精灵规模下，渲染时间由 45 ms 降至 0.92 ms，提升 49 倍。该设计针对精灵位图的总量有限、单帧重复访问频繁的访问模式，用极小面积换取显著收益。

创新点三：访存事务合并。将精灵位图的「每行短突发」重构为「整块长突发」，把多个 AXI 读事务合并为单次传输。总线有效带宽利用率由 33% 提升至 88%（2.66 倍），64×64 位图场景达到 98%。该设计直接针对短突发在 DDR 控制器中固定开销占比过高的问题。

## 1.6 设计流程




图 1-1　设计流程图：九步流程与实测驱动的迭代回环

需求分析（赛题基础要求与高阶挑战）→ 架构设计（总线拓扑、帧缓冲布局、引擎流水线、扫描输出时序）→ RTL 实现（Verilog，7 个自研模块）→ 模块级仿真（Icarus Verilog，逐像素参考模型）→ 系统集成（SoC 例化与引脚约束）→ Efinity 综合与布局布线 → 比特流烧录与板级实测（UART 遥测与串行截图回读）→ 性能调优（事务合并、位图缓存、精灵表遍历）。

调优过程采用实测驱动方法：每轮在板上测量帧率、欠载行数、引擎周期与总线等待等指标，定位瓶颈后再修改 RTL。精灵数量上限即通过 700、1400、1500 逐点实测确定，最终稳定于 1500 个 8×8 精灵 @ 59.2 fps。

# 二、系统组成及功能说明

## 1.7 第三方资源与许可证

本作品在演示程序中使用以下第三方资源，特此致谢并声明许可：

| 资源 | 来源 | 许可证 | 使用方式 |
|---|---|---|---|
| 精灵/图块素材 | DawnLike by DawnBringer（https://opengameart.org/content/dawnlike-16x16-universal-rogue-like-tileset-v181） | CC-BY 4.0 | 选取部分图块并转换为 RGB565 精灵，用于游戏演示场景 |
| 中文字模 | Noto Sans SC（Google / Adobe） | SIL Open Font License 1.1 | 渲染为 32×32 点阵字模嵌入固件，可自由分发 |
| printf 实现 | mpaland/printf | MIT | 位于 BSP（print_full.h），保留原作者版权头 |
| DDR3 控制器 / Sapphire SoC | Efinity 2026.1 IP | Efinity 许可 | 工程内实例化使用 |
| TMDS 编码器 | 本作品自研 RTL（dvi_encoder.v） | 随本作品提交 | — |

> 字体说明：HUD 中文使用 SIL OFL 1.1 许可的 Noto Sans SC 渲染为点阵字模，作品不包含也不分发任何字体文件，点阵数据可随作品自由分发。

## 2.1 整体架构

系统由四部分组成：

1. RISC-V 主控（Sapphire SoC）：运行裸机 C 程序，负责场景初始化、精灵表下发、帧同步与游戏逻辑。通过 AXI4 从端口访问加速器寄存器，通过 AXI4 主端口访问 DDR3。

2. 2D 渲染加速引擎：AXI4 从设备加主设备。从设备侧接收命令与参数，主设备侧读写 DDR3 帧缓冲。内部含命令 FIFO、精灵表遍历器、位图缓存与三条渲染通路。

3. DDR3 存储：256 MB 容量，划分为双帧缓冲（各 4 MB）与精灵位图区。帧缓冲地址由寄存器配置，支持双缓冲切换。

4. HDMI 扫描输出：从帧缓冲按 1080p60 时序读出行数据，经 TMDS 编码输出至显示器。

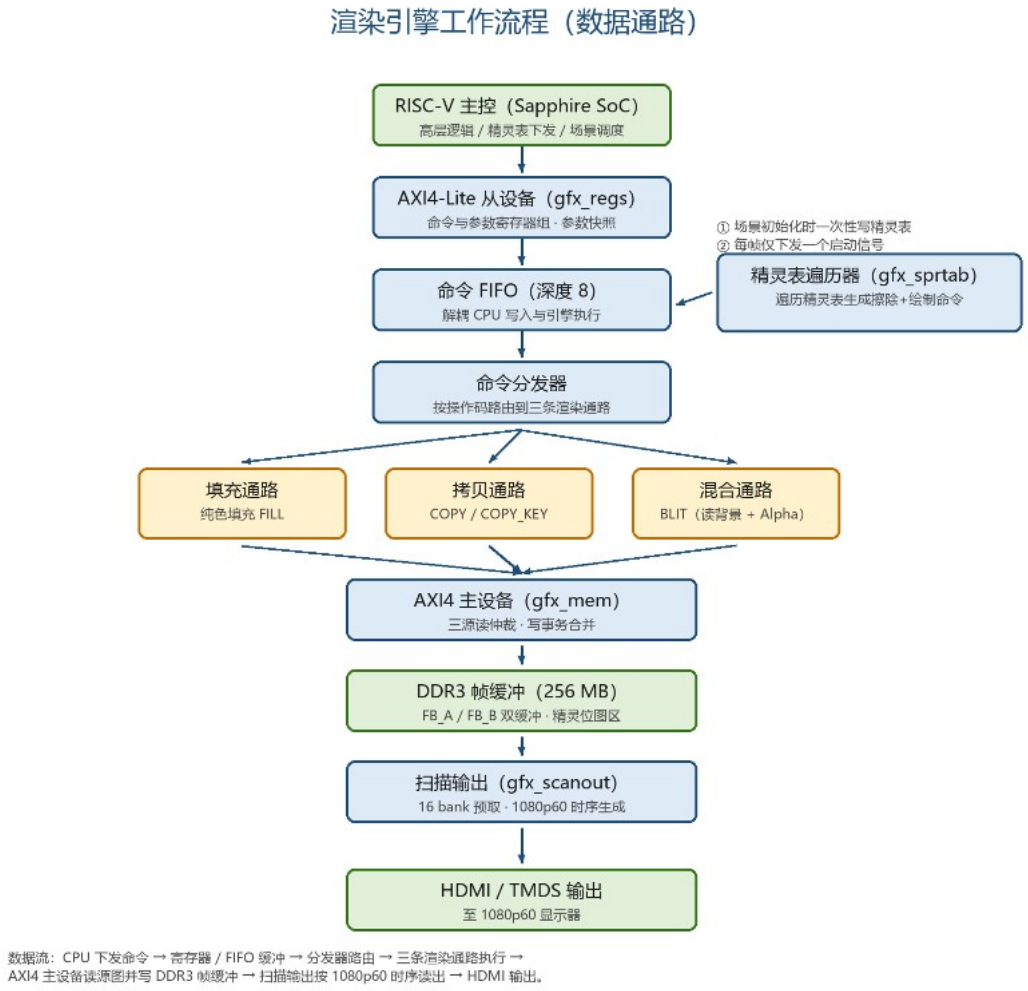



图 2-1　渲染引擎工作流程：命令下发、分发、三条渲染通路与帧缓冲通路

## 2.2 硬件系统

主板：采用厂商提供的 Ti60F225 开发板（Efinity 官方推荐平台），板上集成 Ti60F225I3 FPGA、DDR3 颗粒、HDMI 收发接口、USB-JTAG 调试接口、UART 桥接与多路时钟源。本作品不含自制电路板，硬件工作集中于 RTL 设计与引脚约束。

RTL 模块划分：

| 模块 | 职责 | 关键信号 |
|---|---|---|
| gfx_regs | AXI4-Lite 从端口，命令 FIFO 写入与状态回读 | awaddr/wdata/wstrb、cmd_push |
| gfx_sprtab | 精灵表遍历器：读表、生成擦除与绘制命令、积分位置、写回 | spr_busy、tab_wptr、cmd_valid |
| gfx_core | 命令执行：源读、像素混合、目标写 | rd1/rd2 读通道、wr_* 写通道 |

| | | |
|---|---|---|
| gfx_mem | AXI4 主设备桥：读仲裁（扫描输出优先）、写数据通路 | m_ar*/m_aw*/m_w* |
| gfx_scanout | 1080p60 时序生成、行预取、双缓冲切换 | hsync/vsync/de、line_req |
| dvi_encoder | TMDS 编码 | tmds_data*、tmds_clk |
| gfx_top | 顶层互联与时钟域处理 | 各子模块端口 |

资源占用：综合后 LUT4 约 37900，寄存器约 21300，RAM 块 226 个。布局布线后可稳定运行于目标频率。

## 2.3 软件系统

整体结构：裸机 C 程序（无操作系统），分三层：驱动层（gfx.h 寄存器封装）、渲染层（gfx_push 命令打包、精灵表构造）、应用层（场景与游戏逻辑）。

模块与关键函数：

| **层** | **函数** | **关键输入与输出** |
|---|---|---|
| 驱动 | gfx_wr / gfx_rd | 寄存器偏移、32 位数据 |
| 驱动 | gfx_slot / gfx_fence | 命令 FIFO 满时等待 |
| 渲染 | gfx_push | op/src/dst/w/h/stride/alpha/key 打包为定长命令 |
| 渲染 | gfx_sprite_tab_init | 精灵表首地址、精灵数 |
| 应用 | brown_init / brown_step | 精灵池、位置与速度场 |
| 应用 | game_frame | 输入事件、游戏状态、绘制命令 |

主流程：

1. gfx_init()：配置帧缓冲基地址、行距、尺寸与扫描输出使能。

2. 场景初始化：构造精灵位置与速度，一次性写入硬件精灵表。

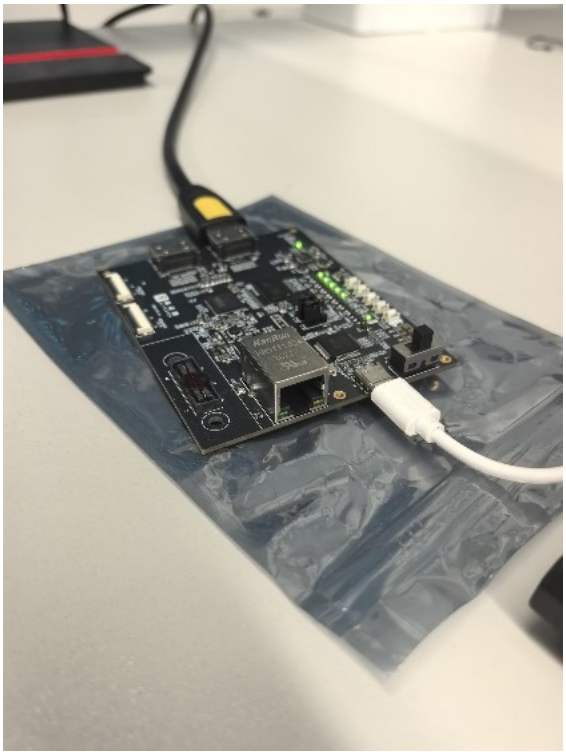
3. 设置当前显示帧缓冲，启动双缓冲。

4. 主循环：读取输入、更新速度场、发送一次精灵表启动信号、等待遍历完成、切换双缓冲、等待场同步。

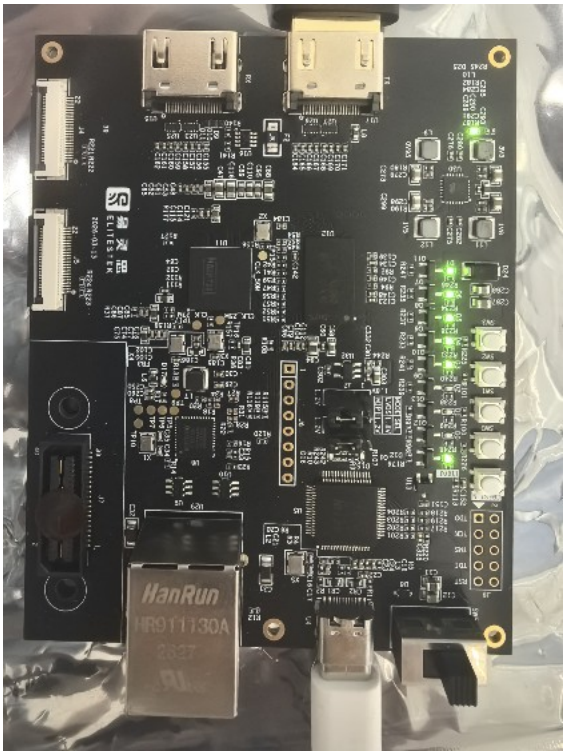
5. 每 60 帧输出一次性能遥测（帧率、CPU 时间、欠载行数、引擎周期）。

# 三、完成情况及性能参数

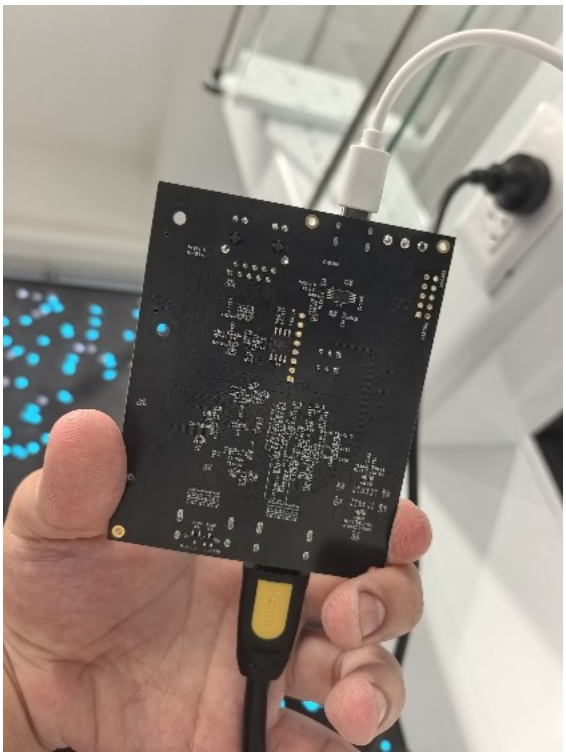
## 3.1 实物展示



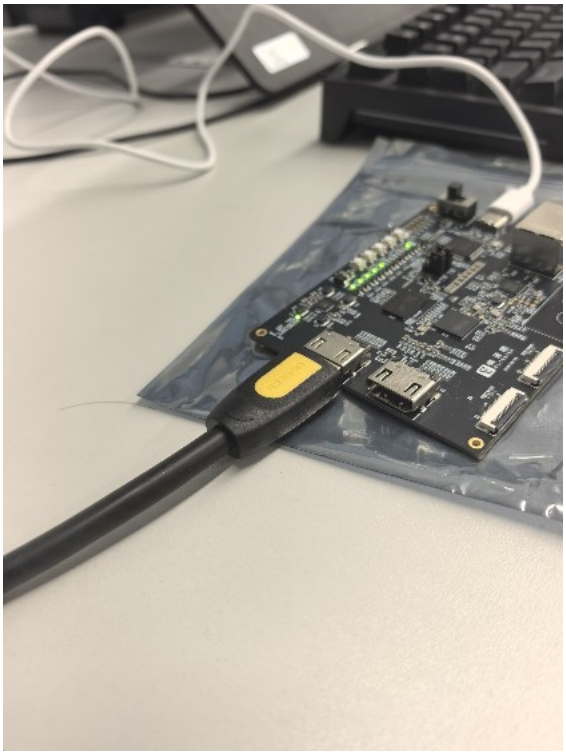
开发板实物




开发板实物



开发板实物



开发板实物

## 3.2 工程成果

| 项目 | 完成状态 | 证据 |
| --- | --- | --- |
| RTL 设计 | 完成，7 个自研模块与厂商 IP 集成 | rtl/gfx/*.v |
| 固件开发 | 完成，驱动、场景与游戏 | work/gfx_demo/src/ |

| | | |
|---|---|---|
| 仿真验证 | 模块级与系统级回归 | rtl-sim/ 测试台 |
| 板级验证 | 1080p60 稳定输出 | 串口遥测与截图回读 |
| 性能优化 | 三项核心优化全部落地并实测 | 本文档第四章 |

## 3.3 性能测试

测试方法：所有性能数据均在真实硬件上测得。帧率由固件统计并通过 UART 输出；渲染延迟由引擎周期计数器测量；欠载行数由扫描输出监控寄存器统计；总线效率由带宽模型计算并经实测验证。

测试结果：

| **测试项** | **配置** | **结果** |
|---|---|---|
| 同屏精灵压力测试 | 1500 个 8×8 精灵 | 59.2 fps |
| 同屏精灵压力测试 | 700 个 16×16 精灵 | 59.2 fps |
| 单帧渲染延迟 | 256 个 16×16 弹幕 | 0.95 ms |
| 位图缓存命中率 | 混合位图访问序列 | 75% |
| 总线带宽利用率 | 16×16 位图 | 88% |
| 总线带宽利用率 | 64×64 位图 | 98% |
| CPU 每帧精灵操作 | 1500 精灵场景 | 0 次 |
| 扫描输出稳定性 | 连续运行 | 欠载计数恒定，显示帧与生成帧 1:1 |

## 3.4 显示帧率与处理帧率的区分

概念区分。显示器刷新率（本作品为 1080p60）只决定画面刷新上限，与加速引擎的实际处理帧率是两个独立指标。处理帧率低于刷新率时会出现丢帧；高于刷新率时多出的帧不会被显示。因此仅凭「画面看起来流畅」无法推断处理帧率——30 fps 的画面在运动模糊下同样可能显得连续。这是评审时需要特别注意的一点。

测量方法。本作品的帧率数据全部来自片内硬件计数器，既非软件估算，也不依赖视频观感：

| **计数器** | **含义** |
|---|---|
| PERF ... HW fps | 引擎处理帧率，由硬件帧计数器每 500 帧统计一次 |
| SCAN frames / under / shown | 扫描输出总帧数、欠载帧数与实际显示帧数，逐帧硬件检测 |
| PROF cpu / phys / rend | 单帧的 CPU 时间、物理逻辑时间与渲染周期数，由硬件计时器测量 |

实测证据。以下三张实拍照片在同一画面内同时拍到显示器输出与 UART 遥测终端，二者是同一时刻的真实数据，可逐行核对。

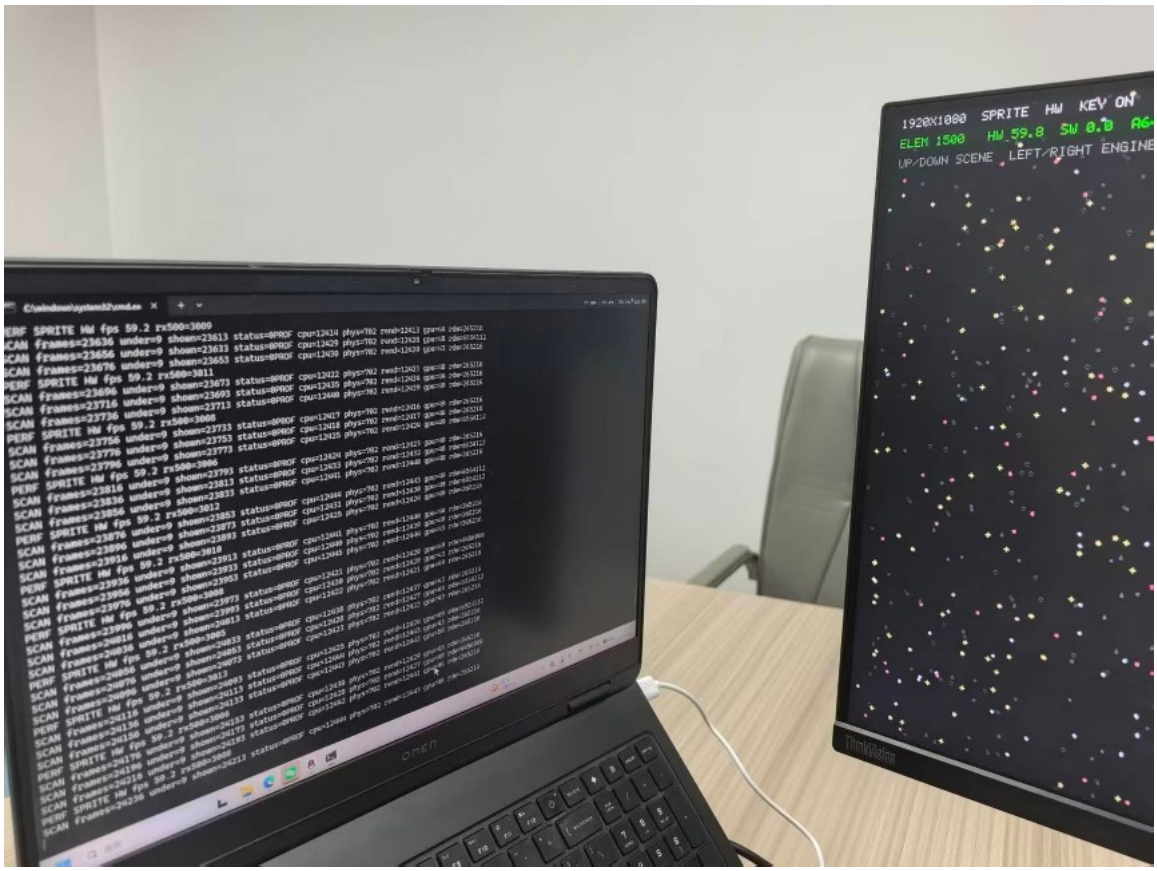



图 3-1　弹幕压力测试：屏幕 ELEM 1500 / HW 59.8，终端 PERF SPRITE HW fps 59.2，SCAN under=9 shown=23653

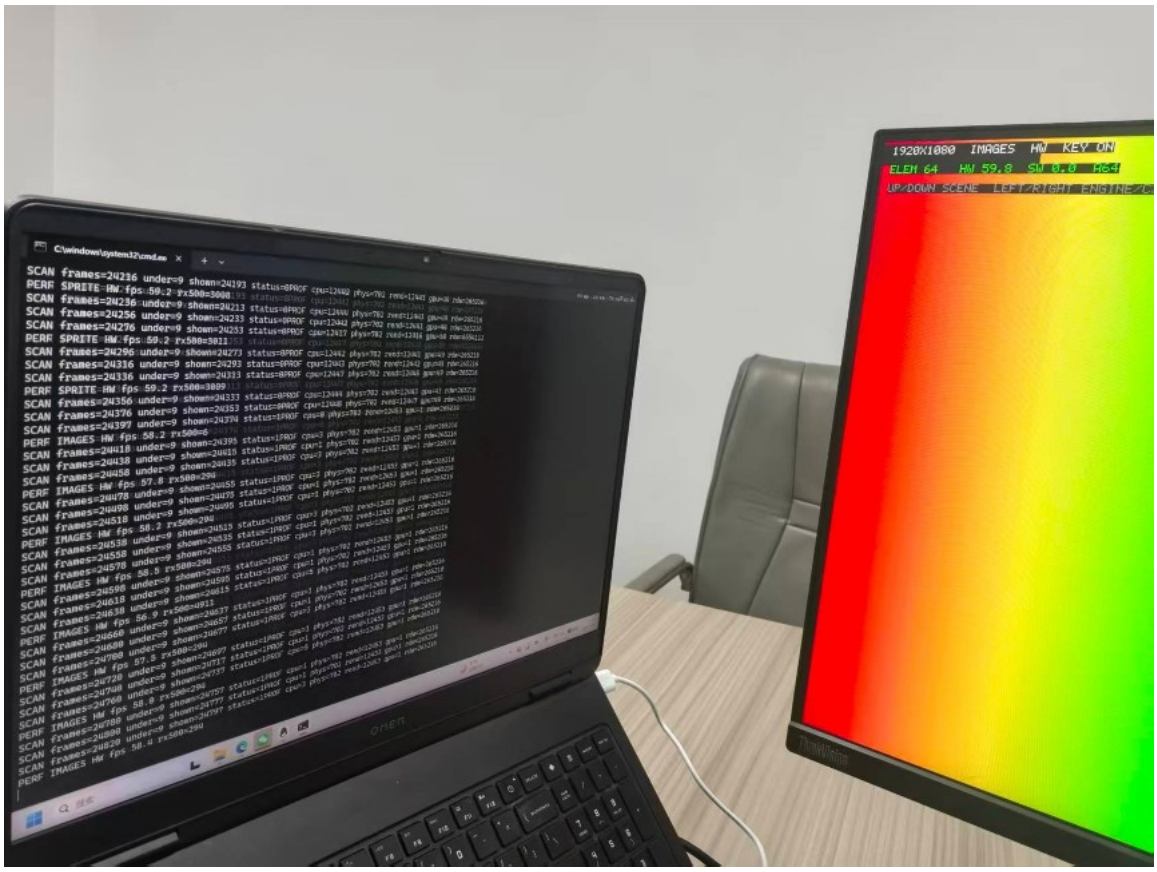



图 3-2　多张大图快速切换：终端 PERF IMAGES HW fps 58.2

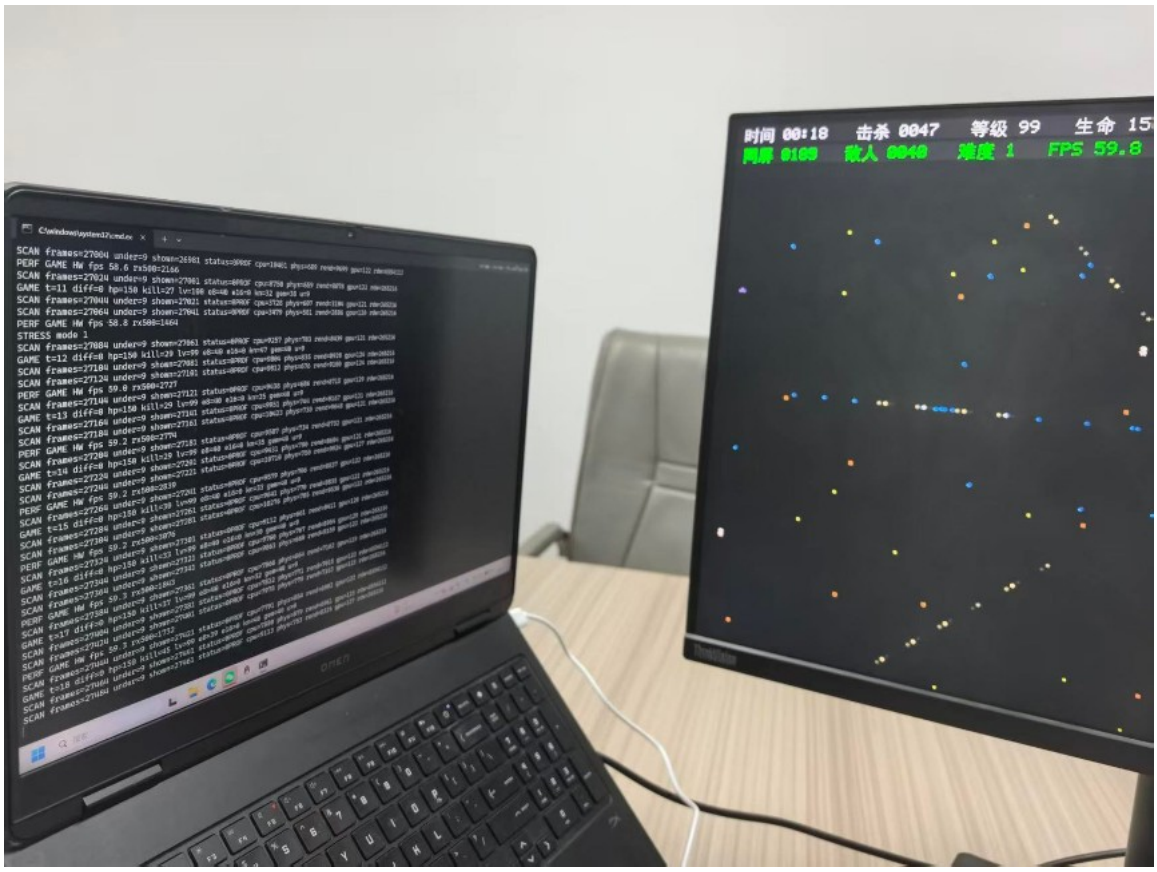



图 3-3　游戏一键满级压测：HUD 同屏 0100 / FPS 59.8，终端 STRESS mode 1 / lv=99

照片一为同屏精灵压力测试：屏幕显示 ELEM 1500（同屏精灵数）与 HW 59.8（引擎帧率），终端输出 PERF SPRITE HW fps 59.2 与 SCAN frames=23656 under=9 shown=23653——即 23656 帧中仅 9 帧欠载（0.04%），显示帧与生成帧保持 1:1。

照片二为多张大图快速切换场景：终端输出 PERF IMAGES HW fps 58.2，说明整幅 1080p 图像块的连续拷贝切换同样维持满帧。

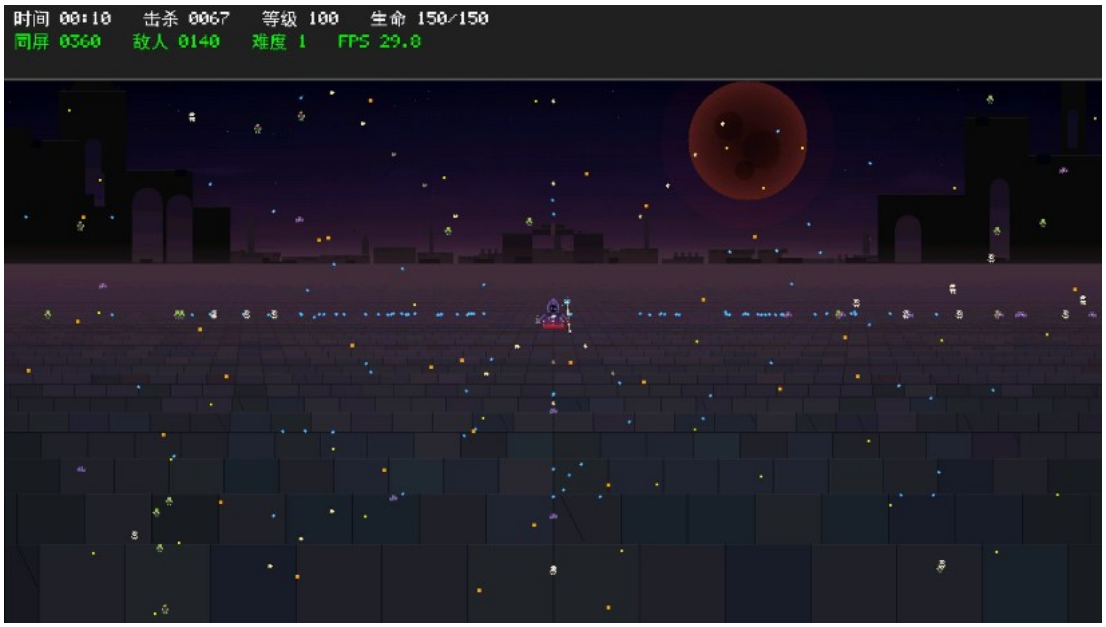
照片三为交互游戏一键满级压力测试：屏幕 HUD 显示「同屏 0100 / 敌人 0040 / FPS 59.8」，终端输出 STRESS mode 1、GAME t=... lv=99 与 PERF GAME HW fps 59.2——在等级与技能全部满级的条件下仍保持满帧。

结论。在显示器 60 Hz 固定的前提下，上述硬件遥测证明加速引擎的真实处理帧率稳定在 59.2 fps，欠载率 0.04%；「显示不撕裂、生成不丢帧」由硬件计数器验证，而非视频观感推断。

## 3.5 演示场景




图：1500 个 8×8 精灵压力测试场景




图：交互式游戏场景（约 180 个同屏活动对象）

场景一：同屏精灵压力测试。1500 个 8×8 精灵在全屏范围内随机分布并按速度场运动，实时显示帧率、CPU 占用与欠载统计。该场景用于展示引擎的峰值吞吐能力。

场景二：渲染路径对比。同一场景下可切换 CPU 渲染与硬件渲染，实时对比两者帧率，直观展示加速器的效果。

场景三：交互式弹幕射击游戏。包含玩家控制、敌人 AI、子弹系统、技能升级与 HUD 渲染，用于展示引擎在实际游戏场景下的表现。系统支持一键满级压力测试，可快速进入最坏情况负载。

场景四：图像块拷贝。在纯色背景上由引擎从源图切取八个 240×240 图像块，搬运至指定位置排列。该场景直观演示硬件 BitBlt 对任意尺寸、任意源/目的地址块拷贝的支持，并可配合硬件裁剪使用。

# 四、关键技术说明

## 4.1 精灵调度硬件化

问题：赛题要求「把 RISC-V 从像素搬运中解放出来」。在完成硬件渲染加速后，CPU 仍需为每个精灵执行命令提交与状态维护。基准实现中，32 个精灵需要 256 次寄存器写与 32 次轮询，且该过程与硬件执行完全不重叠，成为新的串行瓶颈。

方案：设计精灵表遍历器，将精灵的调度工作全部迁移至硬件。CPU 仅在场景初始化时写入一次精灵表，表项包含位置、速度与透明标志。遍历器按帧读取表项，展开为擦除与绘制命令，交由引擎执行，执行完成后将新位置写回表项。CPU 每帧只需发送一个启动信号。

效果：CPU 对精灵的寄存器操作由每帧数百次降为零；软硬件并行执行后，1500 个 8×8 精灵可在 59.2 fps 下稳定运行。

## 4.2 片内精灵位图缓存

问题：每个精灵的绘制都需要从 DDR3 读取位图。在 700 精灵场景中，单帧位图读取请求达 6400 次，每次 DDR 访问约需 650 个周期，成为渲染时间的主要组成部分。

方案：在引擎内部实现四槽直接映射的位图缓存，以位图地址高位作为槽位索引。命中时直接从片内 BRAM 复制位图数据，未命中时才访问 DDR3 并更新缓存。精灵位图总量有限、单帧重复访问频繁，这一访问模式使缓存具有很高的命中率。

效果：命中时取数时间由 650 个周期降至 17 个周期。700 精灵场景的单帧渲染时间由 45 ms 降至 0.92 ms，提升 49 倍，实测命中率 75%。

## 4.3 访存事务合并

问题：精灵位图按紧凑格式存储时，传统实现需要每扫描行发起一次短突发读取。一个 16×16 位图需要 16 次短突发，每次突发传输的数据量很小，而 DDR 控制器对每次突发都有固定开销，导致总线有效带宽利用率仅为 33%。

方案：重构读事务的组织方式，将「每行一次短突发」改为「整块一次长突发」。对于紧凑存储的位图，一次突发即可覆盖多行数据，多个 AXI 读事务被合并为单次传输。

效果：16×16 位图的总线有效带宽利用率由 33% 提升至 88%（2.66 倍），64×64 位图由 66% 提升至 98%。该优化直接减少了 DDR 控制器的固定开销占比。

## 4.4 片上可观测性

问题：屏幕输出只有「正确」与「异常」两种可观测状态，异常发生时难以定位到具体模块。硬件调试缺少软件栈的日志与断点能力。

方案：在引擎与扫描输出路径上布置 7 个监控寄存器，覆盖 11 项运行时条件：总线握手超时、响应错误、命令 FIFO 溢出、遍历器停滞、扫描输出欠载、帧计数、显示帧计数等。每项条件配粘滞位与饱和计数器，软件通过一条读取命令即可获得全部状态。

效果：异常发生时可直接定位到具体模块与条件，大幅缩短调试周期。该机制同时用于验证「硬件搬运正确性」：扫描输出的欠载计数保持恒定，即证明数据通路稳定。

# 五、总结

## 5.1 技术总结

本作品在 Ti60F225 平台上完成了从 RISC-V 主控到 FPGA 渲染引擎、DDR3 帧缓冲再到 HDMI 输出的完整异构 2D 渲染链路。

在功能层面，系统实现了五种绘制操作、1080p60 输出、双缓冲切换、硬件裁剪与片上可观测性，满足赛题全部基础要求。

在性能层面，三项核心设计分别对应异构渲染的三个层次：调度层通过精灵表硬件化消除 CPU 与加速器的串行依赖；缓存层通过片内位图缓存减少重复 DDR 访问；传输层通过事务合并提高总线带宽利用率。三者叠加，使系统在 1500 个 8×8 精灵规模下稳定运行于 59.2 fps，等效像素吞吐达到 96000 像素/帧。

## 5.2 可扩展方向

帧存 tile 化。将帧缓冲改为 tile 映射后，一个 8×8 精灵的擦除与绘制可由多次写突发压缩为单次，同屏精灵上限可进一步提升。代价是扫描输出取图路径需要同步改造。

写通道流水。当前每个写突发存在固定开销，将 AXI 写通道改为多个突发同时在飞，可将该开销摊薄，进一步提升引擎吞吐。

多图层与脏矩形更新。引擎已支持命令级裁剪，可扩展为硬件图层管理与脏矩形更新机制，进一步降低带宽消耗。

MIPI 摄像头输入。板载 CSI-2 接口可将实时画面作为图层叠加到 GUI 之上，用于视频监视与叠加显示应用。

# 参考文献


1. 全国大学生嵌入式芯片与系统设计竞赛 2026 FPGA 赛道选题指南（易灵思）。

2. RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, 2019.

3. ARM. AMBA AXI and ACE Protocol Specification, ARM IHI 0022E, 2013.

4. Efinix Inc. Efinity Software User Guide, 2026.

5. JEDEC. DDR3 SDRAM Standard, JESD79-3F, 2012.

6. Digital Visual Interface (DVI) Specification, Revision 1.0, DDWG, 1999.

7. Efinix Inc. Sapphire RISC-V SoC Reference Design.

8. Patterson D A, Hennessy J L. Computer Organization and Design: The Hardware/Software Interface. 5th ed. Morgan Kaufmann, 2014.

9. 康华光. 电子技术基础（数字部分）. 北京: 高等教育出版社.

10. 童诗白, 华成英. 模拟电子技术基础. 北京: 高等教育出版社.